Ensemble Methods

Thanh-Sach LE

✉ ltsach@hcmut.edu.vn

Faculty of Computer Science & Engineering
Ho Chi Minh City University of Technology (HCMUT), VNU-HCM

Tp.HCM on November 26, 2025

Content

Ensemble Methods: Motivation and Overview

Bagging (Bootstrap Aggregating)

Random Forest

Boosting

Stacking and Voting

Comparison and Practical Tips

Content

1. Ensemble Methods: Motivation and Overview
2. Bagging (Bootstrap Aggregating)
3. Random Forest
4. Boosting
5. Stacking and Voting
6. Comparison and Practical Tips

1 Content

Ensemble Methods: Motivation and Overview

Bagging (Bootstrap Aggregating)

Random Forest

Boosting

Stacking and Voting

Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Ensemble Methods: Motivation and Overview

Content
2 Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
Random Forest
Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Why Ensemble Methods?

- **Single model:** Limited capacity, sensitive to the particular training sample.
- **Problem:** May suffer from high variance, high bias, or both.
- **Key idea:** Combine multiple models (learners) to obtain a more robust predictor.
- **Analogy:** "Wisdom of the crowd" – averaging many opinions can be more accurate than any individual opinion.

Content
3 Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
Random Forest
Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bias–Variance Perspective

- **Bias:** Error due to wrong assumptions about the true function (underfitting).
- **Variance:** Error due to sensitivity to fluctuations in the training data (overfitting).
- **Ensemble goal:**
  - Reduce variance by **averaging** predictions of multiple models.
  - Sometimes also reduce bias by **iteratively** adding stronger models.
- Many ensemble methods exploit the fact that:

$$\mathrm{Var}\Big(\frac{1}{M}\sum_{m=1}^{M}\hat{y}^{(m)}\Big) \approx \frac{1}{M^2}\sum_{m=1}^{M}\mathrm{Var}(\hat{y}^{(m)})$$

when models are weakly correlated.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bagging (Bootstrap Aggregating)

Content

Ensemble Methods: Motivation and Overview

5 Bagging (Bootstrap Aggregating)

Random Forest

Boosting

Stacking and Voting

Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bagging: Intuition

- **Goal:** Reduce variance of an unstable base learner (e.g., decision trees).
- **Bootstrap sample:** From training set of size n, sample n points *with replacement*.
- **Procedure:**
  - Train many models on different bootstrap samples.
  - Aggregate their predictions by averaging (regression) or voting (classification).
- **Effect:** Randomness in the training data for each model decorrelates errors, leading to variance reduction.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bagging in scikit-learn: Toy Example (1/2)

- Use `BaggingClassifier` with a decision tree as base learner.
- Train on a toy dataset and compare single tree vs bagging.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

X, y = make_moons(n_samples=500, noise=0.3, random_state=0)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

# Single decision tree (high variance)
tree = DecisionTreeClassifier(max_depth=None, random_state=0)
tree.fit(X_train, y_train)
y_pred_tree = tree.predict(X_test)
```


Content
Ensemble Methods: Motivation and Overview
7 Bagging (Bootstrap Aggregating)
Random Forest
Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bagging in scikit-learn: Toy Example (2/2)

- Add bagging on top of decision trees.
- Compare accuracies on the test set.

```
from sklearn.ensemble import BaggingClassifier

bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=100,
    bootstrap=True,
    n_jobs=-1,
    random_state=0
)
bagging.fit(X_train, y_train)
y_pred_bag = bagging.predict(X_test)

print("Tree accuracy:    ", accuracy_score(y_test, y_pred_tree))
print("Bagging accuracy:", accuracy_score(y_test, y_pred_bag))
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Bagging Algorithm

Algorithm 1 Bagging (Bootstrap Aggregating)

1: Given training set $D = \{(x_i, y_i)\}_{i=1}^{n}$, base learner $\mathcal{L}$, number of models M
2: **for** $m = 1, \ldots, M$ **do**
3: Sample bootstrap dataset D_m by drawing n samples with replacement from D
4: Train base model $h_m = \mathcal{L}(D_m)$
5: **end for**
6: For a new input x:

Regression: $\hat{y}(x) = \frac{1}{M} \sum_{m=1}^{M} h_m(x)$

Classification: $\hat{y}(x) = \text{majority_vote}\big(h_1(x), \ldots, h_M(x)\big)$

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bagging: Properties

- **Best for:** High-variance, low-bias models (e.g., full decision trees).
- **Pros:**
  - Simple to implement.
  - Parallelizable: all models can be trained independently.
  - Strong reduction in variance.
- **Cons:**
  - Large storage cost (many models).
  - Slower prediction than a single model.
  - Harder to interpret than one tree.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Random Forest

Content

Ensemble Methods: Motivation and Overview

Bagging (Bootstrap Aggregating)

11 Random Forest

Boosting

Stacking and Voting

Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Random Forest: Bagging + Random Features

- **Base learner:** Decision tree.
- **Two sources of randomness:**
  - Bootstrap sampling of the training data for each tree.
  - Random subset of features considered at each split.
- **Motivation:** Reduce correlation between trees to obtain stronger variance reduction.
- **Prediction:** Average (regression) or majority vote (classification) over all trees.

Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
12 Random Forest
Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Random Forest: Training Algorithm

Algorithm 2 Random Forest Training

1: Given training set $D = \{(x_i, y_i)\}_{i=1}^{n}$ with d features, number of trees M
2: **for** $m = 1, \ldots, M$ **do**
3: **Step 1:** Sample bootstrap dataset D_m by drawing n samples with replacement from D
4: **Step 2:** Build decision tree T_m on D_m:
While stopping criterion not met:
Randomly select subset of features $\mathcal{F}_{\text{sub}}$ (typically $\sqrt{d}$ for classification)
For each feature in $\mathcal{F}_{\text{sub}}$:
Find best split point (e.g., maximize information gain)
Choose best split among $\mathcal{F}_{\text{sub}}$ (not all features)
Split node into child nodes
5: **end for**
6: **Step 3:** Return ensemble $\{T_1, T_2, \ldots, T_M\}$

Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
13 Random Forest
Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Random Forest: Step-by-Step Training Process

For each tree $m = 1, \ldots, M$:

1. **Bootstrap Sampling:**
   - Randomly sample n examples **with replacement** from training set D
   - Result: Bootstrap dataset D_m (some examples may appear multiple times, others not at all)
   - About 63.2% of original examples appear in each bootstrap sample (on average)
2. **Build Decision Tree on D_m:**
   - Start with root node containing all examples in D_m
   - **Recursively split nodes** until stopping criterion met:

Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
14 Random Forest
Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Random Forest: Step-by-Step Training Process (Cont.)

At each split node:

❸ Random Feature Selection:

- Randomly select k features from d total features
- $k = \sqrt{d}$ (classification) or $k = d/3$ (regression)
- This subset: $\mathcal{F}_{\text{sub}} = \{f_1, f_2, \ldots, f_k\}$

❹ Find Best Split:

- For each feature $f_j \in \mathcal{F}_{\text{sub}}$:
  - Evaluate all possible split points (e.g., for continuous features)
  - Compute split criterion: Information Gain, Gini Impurity, or MSE
- Choose the feature and split point that gives **best criterion value**
- **Note:** Only considers features in $\mathcal{F}_{\text{sub}}$, not all d features!

❺ Split Node:

- Create child nodes based on chosen split
- Continue recursively for each child node

Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
15 Random Forest
Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Random Forest: Training Summary

- At each split in a tree:
  - Randomly select a subset of features $\mathcal{F}_{\text{sub}}$.
  - Find the best split using only $\mathcal{F}_{\text{sub}}$.
- **Typical choice:**
  - Classification: $|\mathcal{F}_{\text{sub}}| = \sqrt{d}$.
  - Regression: $|\mathcal{F}_{\text{sub}}| = \frac{d}{3}$.
- **Key hyperparameters:**
  - Number of trees M.
  - Maximum depth of each tree.
  - Minimum samples per leaf / split.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Random Forest: Pros and Cons

- **Pros:**
  - Strong performance on many tabular datasets.
  - Handles mixed feature types and non-linear interactions.
  - Built-in estimate of feature importance.
- **Cons:**
  - Less interpretable than a single decision tree.
  - Large memory footprint and slower inference.
  - May underperform gradient boosting on highly structured/tabular data.

Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
17 Random Forest
Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Boosting

Content

Ensemble Methods: Motivation and Overview

Bagging (Bootstrap Aggregating)

Random Forest

18 Boosting

Stacking and Voting

Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Boosting: Main Idea

- **Sequential ensemble:** Train models one after another.
- Each new model focuses on **examples that previous models predicted poorly**.
- Base learners are typically **weak** (e.g., shallow trees).
- **Contrast with bagging:**
  - Bagging: models are trained independently and can be parallelized.
  - Boosting: models are dependent and training is sequential.

Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
Random Forest
19 Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdaBoost: Intuition

- Consider binary classification $y_i \in \{-1, +1\}$.
- Maintain a **weight** w_i for each training example.
- Train a weak learner on the weighted dataset.
- Increase the weights of misclassified examples and decrease weights of correctly classified ones.
- Final classifier is a **weighted majority vote** of weak learners.

Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
Random Forest
20 Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdaBoost: Algorithm (Simplified)

Algorithm 3 AdaBoost (Binary Classification)

1: Initialize weights $w_i^{(1)} = \frac{1}{n}$ for all $i = 1, \ldots, n$
2: **for** $t = 1, \ldots, T$ **do**
3: Train weak learner h_t using weights $w_i^{(t)}$
4: Compute weighted error $\varepsilon_t = \frac{\sum_{i=1}^{n} w_i^{(t)} \mathbb{I}[h_t(x_i) \neq y_i]}{\sum_{i=1}^{n} w_i^{(t)}}$
5: Compute learner weight $\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$
6: Update weights:

$$w_i^{(t+1)} = w_i^{(t)} \exp\big(-\alpha_t y_i h_t(x_i)\big)$$

7: Normalize $w_i^{(t+1)}$ so that $\sum_i w_i^{(t+1)} = 1$
8: **end for**
9: Final classifier: $H(x) = \text{sign}\left(\sum_{t}^{T} \alpha_t h_t(x)\right)$

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdaBoost: Step 1 – Sample Weights as a Distribution

- At iteration t, we maintain a weight $w_i^{(t)}$ for each training example (x_i, y_i).
- The weights are normalized so that:

$$\sum_{i=1}^{n} w_i^{(t)} = 1,$$

  so they define a probability distribution over training examples.
- **Interpretation:**
  - Large $w_i^{(t)}$: example i is "important" and should be fitted well.
  - Small $w_i^{(t)}$: example i is currently less important.
- Initial step: all examples are equally important,

$$w_i^{(1)} = \frac{1}{n}.$$

AdaBoost: Step 2 – Train Weak Learner and Weighted Error

Trainable using weighted data

- Weak learner must be trainable using `sample_weight`.
- Examples:
  - **Can be trained using weighted data:**
    - Decision Trees (CART), Random Forests, Extra Trees.
    - Gradient Boosted Trees (XGBoost, LightGBM, CatBoost).
    - Logistic Regression (supports weighted log-likelihood).
    - Linear Models (Linear Regression, Ridge, Lasso, Elastic Net).
    - SVMs (SVC, LinearSVC) with weighted hinge loss.
    - Naive Bayes (GaussianNB, MultinomialNB, BernoulliNB).
  - **Cannot be trained using weighted data:**
    - KNN (no training phase, cannot incorporate sample weights).
    - Neural Networks (unless implemented manually to use weights).

Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
Random Forest
23 Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdaBoost: Step 2 – Train Weak Learner and Weighted Error

- Train a weak learner h_t using the current weights $w_i^{(t)}$.
- The learner sees a **weighted** training set: misclassified examples with large weights hurt more.
- Weighted error:

$$\varepsilon_t = \frac{\sum_{i=1}^{n} w_i^{(t)} \mathbb{I}[h_t(x_i) \neq y_i]}{\sum_{i=1}^{n} w_i^{(t)}}.$$

- **Meaning:**
  - Numerator: total weight of misclassified examples.
  - Denominator: total weight of all examples (equal to 1 after normalization).
  - So ε_t is the misclassification rate under the current weight distribution.

AdaBoost: Step 3 – Learner Weight α_t

weak learner + better than random

- **h_t is a weak learner, but it is better than random:** $\varepsilon_t < 0.5$.
- If $\varepsilon_t < 0.5$: then continue training h_t on the next iteration, or stop training.

- Compute

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}.$$

- **h_t is better than random:** $\varepsilon_t < 0.5$:
  - Then $\frac{1-\varepsilon_t}{\varepsilon_t} > 1$ and $\alpha_t > 0$.
  - Smaller ε_t (more accurate h_t) implies larger α_t.
- **Interpretation:**
  - α_t is the "vote weight" of h_t in the final classifier.
  - Stronger weak learners (lower error) receive higher influence.

Ensemble Methods

Content

Ensemble Methods: Motivation and Overview

Bagging (Bootstrap Aggregating)

Random Forest

25 Boosting

Stacking and Voting

Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdaBoost: Step 4 – Update Sample Weights

- Update:
$$w_i^{(t+1)} = w_i^{(t)} \exp\big(-\alpha_t y_i h_t(x_i)\big).$$
- Assume $y_i \in \{-1, +1\}$ and $h_t(x_i) \in \{-1, +1\}$.
- **If h_t predicts correctly:** $y_i h_t(x_i) = 1$
$$w_i^{(t+1)} = w_i^{(t)} e^{-\alpha_t} \quad \Rightarrow \quad \text{weight decreases.}$$
- **If h_t predicts incorrectly:** $y_i h_t(x_i) = -1$
$$w_i^{(t+1)} = w_i^{(t)} e^{+\alpha_t} \quad \Rightarrow \quad \text{weight increases.}$$
- Misclassified examples become more important in the next iteration.

Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
Random Forest
26 Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdaBoost: Step 5 – Renormalization and Final Classifier

- After updating, renormalize:

$$w_i^{(t+1)} \leftarrow \frac{w_i^{(t+1)}}{\sum_{j=1}^{n} w_j^{(t+1)}} \quad \Rightarrow \quad \sum_{i=1}^{n} w_i^{(t+1)} = 1.$$

- At the end, combine all weak learners:

$$H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right).$$

- **View:** $H(x)$ is a **weighted majority vote**:
  - Each h_t casts a vote $h_t(x) \in \{-1, +1\}$.
  - Vote strength is α_t.
- Final decision uses the sign of the weighted sum of all votes.

Ensemble Methods

Content

Ensemble Methods: Motivation and Overview

Bagging (Bootstrap Aggregating)

Random Forest

27 Boosting

Stacking and Voting

Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdaBoost in scikit-learn: Toy Example (1/2)

- Use `AdaBoostClassifier` with shallow decision trees (stumps) as weak learners.
- Train on a toy dataset and compare a single tree vs AdaBoost.

```
1  from sklearn.datasets import make_moons
2  from sklearn.model_selection import train_test_split
3  from sklearn.tree import DecisionTreeClassifier
4  from sklearn.metrics import accuracy_score
5
6  X, y = make_moons(n_samples=500, noise=0.3, random_state=0)
7  X_train, X_test, y_train, y_test = train_test_split(
8      X, y, test_size=0.3, random_state=0
9  )
10
11 # Single shallow decision tree (weak learner)
12 tree = DecisionTreeClassifier(max_depth=1, random_state=0)
13 tree.fit(X_train, y_train)
14 y_pred_tree = tree.predict(X_test)
```


Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
Random Forest
28 Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

AdaBoost in scikit-learn: Toy Example (2/2)

- Wrap the weak learner in `AdaBoostClassifier`.
- Compare test accuracy of the single tree vs AdaBoost.

```
from sklearn.ensemble import AdaBoostClassifier

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=100,
    learning_rate=0.5,
    random_state=0
)
ada.fit(X_train, y_train)
y_pred_ada = ada.predict(X_test)

print("Tree accuracy:      ", accuracy_score(y_test, y_pred_tree))
print("AdaBoost accuracy:  ", accuracy_score(y_test, y_pred_ada))
```

Gradient Boosting: High-Level View

- General framework for regression and classification.
- View boosting as **gradient descent** in function space.
- At iteration t:
  - Compute pseudo-residuals (negative gradient of loss w.r.t. predictions).
  - Fit a new tree h_t to these residuals (**note: h_t is a regression task, even for classification problems**)
  - Update the model: $F_t(x) = F_{t-1}(x) + \eta h_t(x)$
  - η is the **learning rate** (step size).
- Implementations: XGBoost, LightGBM, CatBoost, etc.

Ensemble Methods

Content

Ensemble Methods: Motivation and Overview

Bagging (Bootstrap Aggregating)

Random Forest

30 Boosting

Stacking and Voting

Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Gradient Boosting: Model and Loss

- Training data: $D = \{(x_i, y_i)\}_{i=1}^{n}$.
- Model is an additive function:

$$F_T(x) = \sum_{t=1}^{T} \eta_t h_t(x),$$

  where each h_t is a weak learner (e.g., regression tree).
- We minimize a differentiable empirical loss:

$$\mathcal{L}(F) = \sum_{i=1}^{n} \ell\big(y_i, F(x_i)\big).$$

- **Idea:** At step t, move F_{t-1} in the direction of the **negative gradient** of $\mathcal{L}$ with respect to F.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Gradient Boosting for Regression (Squared Error)

- Loss for regression:

$$\ell\big(y_i, F(x_i)\big) = \frac{1}{2}\big(y_i - F(x_i)\big)^2.$$

- Gradient with respect to prediction $F(x_i)$:

$$\frac{\partial \ell}{\partial F(x_i)} = F(x_i) - y_i.$$

- Negative gradient (pseudo-residual):

$$r_i^{(t)} = -\frac{\partial \ell}{\partial F(x_i)}\Big|_{F=F_{t-1}} = y_i - F_{t-1}(x_i).$$

- **Algorithm step:**
  - Compute residuals $r_i^{(t)} = y_i - F_{t-1}(x_i)$.
  - Fit a regression tree h_t to data $\{(x_i, r_i^{(t)})\}$.
  - Update: $F_t(x) = F_{t-1}(x) + \eta h_t(x)$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Gradient Boosting for Binary Classification (Logistic Loss)

- Assume $y_i \in \{-1, +1\}$ and model $F(x)$ outputs a real score.
- Use logistic (binomial deviance) loss:

$$\ell\big(y_i, F(x_i)\big) = \log\big(1 + \exp(-2y_i F(x_i))\big).$$

- Gradient with respect to $F(x_i)$:

$$\frac{\partial \ell}{\partial F(x_i)} = -\frac{2y_i}{1 + \exp\big(2y_i F(x_i)\big)}.$$

- Negative gradient (pseudo-residual):

$$r_i^{(t)} = \frac{2y_i}{1 + \exp\big(2y_i F_{t-1}(x_i)\big)}.$$

- **Algorithm step:**
  - Compute $r_i^{(t)}$ for all training points.
  - Fit a regression tree h_t to $(x_i, r_i^{(t)})$.
  - Update: $F_t(x) = F_{t-1}(x) + \eta h_t(x)$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Boosting: Pros and Cons

- **Pros:**
  - Often achieves state-of-the-art performance on tabular data.
  - Can reduce both bias and variance.
  - Flexible choice of loss functions.
- **Cons:**
  - More sensitive to noise and outliers than bagging.
  - Requires careful tuning (number of trees, depth, learning rate).
  - Training is sequential and less parallelizable.

Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
Random Forest
34 Boosting
Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Stacking and Voting

Content

Ensemble Methods: Motivation and Overview

Bagging (Bootstrap Aggregating)

Random Forest

Boosting

35 Stacking and Voting

Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Simple Voting and Averaging

- **Voting classifier:** Combine several classifiers by majority (hard) or weighted (soft) voting.
- **Averaging regressor:** Average predictions of several regressors.
- **Use case:** Combine strong but diverse models (e.g., SVM, random forest, neural network).
- Simple but often effective baseline ensemble.

ltsach@hcmut.edu.vn
Thanh-Sach LE

VotingClassifier in scikit-learn (1/3)

- Combine different classifiers (logistic regression, decision tree, SVM).
- Use soft voting (average predicted probabilities).

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(
    n_samples=500, n_features=10,
    n_informative=5, n_redundant=0,
    random_state=0
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)
```

VotingClassifier in scikit-learn (2/3)

- Define the base classifiers.

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.svm import SVC
4
5 clf1 = LogisticRegression(max_iter=1000, random_state=0)
6 clf2 = DecisionTreeClassifier(max_depth=5, random_state=0)
7 clf3 = SVC(probability=True, random_state=0)
```

VotingClassifier in scikit-learn (3/3)

- Create a `VotingClassifier` with soft voting.
- Compare accuracy on the test set.

```
from sklearn.ensemble import VotingClassifier
from sklearn.metrics import accuracy_score

voting_clf = VotingClassifier(
    estimators=[('lr', clf1), ('dt', clf2), ('svm', clf3)],
    voting='soft'
)

voting_clf.fit(X_train, y_train)
y_pred = voting_clf.predict(X_test)

print("Voting accuracy:", accuracy_score(y_test, y_pred))
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Stacking (Stacked Generalization)

- **Level-0 models (base learners):** Train multiple different models on the same training data.
- **Level-1 model (meta-learner):** Takes as input the predictions of level-0 models and outputs the final prediction.
- **Training strategy:**
  - Use cross-validation to generate out-of-fold predictions from base learners.
  - Use these predictions (and possibly original features) to train the meta-learner.
- **Advantage:** Learns how to optimally combine heterogeneous models.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Stacking: Architecture and Notation

- Suppose we have M base learners (level-0):

$$h_1(x), h_2(x), \ldots, h_M(x).$$

- For each input x, base learners produce predictions:

$$z(x) = \big(h_1(x), \ldots, h_M(x)\big).$$

- The meta-learner (level-1) is a model g trained on $z(x)$ (and optionally original x):

$$\hat{y}(x) = g\big(z(x)\big) \quad \text{or} \quad \hat{y}(x) = g\big(x, z(x)\big).$$

- **Key idea:** Learn how to combine base predictions instead of using fixed rules (as in voting).

Ensemble Methods
Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
Random Forest
Boosting
41 Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Stacking: Training with Cross-Validation

- Directly training g on predictions of h_m from the same data causes overfitting.
- Solution: use K-fold cross-validation to get **out-of-fold** predictions.
- For each base learner h_m:
  - Split training data into K folds.
  - For each fold k: train h_m on other $K-1$ folds, predict on fold k.
  - Concatenate predictions over all folds to get out-of-fold vector

$$z_{i,m} = h_m^{(-k)}(x_i),$$

where x_i belongs to fold k.

Ensemble Methods
Content
Ensemble Methods: Motivation and Overview
Bagging (Bootstrap Aggregating)
Random Forest
Boosting
42 Stacking and Voting
Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Stacking: Training the Meta-Learner

- After obtaining out-of-fold predictions $\{z_{i,m}\}$ for all base learners $m = 1, \ldots, M$:
  - Construct a new feature vector for each training example:

$$z_i = (z_{i,1}, \ldots, z_{i,M}).$$

  - The corresponding target remains y_i.
- **Meta-training dataset:**

$$\{(z_i, y_i)\}_{i=1}^{n}.$$

- Choose any suitable supervised learner as meta-model g:
  - Classification: logistic regression, linear SVM, random forest, etc.
  - Regression: linear regression, ridge, random forest regressor, etc.
- Train g on (z_i, y_i) exactly as in a standard supervised learning problem.

Ensemble Methods

Content

Ensemble Methods: Motivation and Overview

Bagging (Bootstrap Aggregating)

Random Forest

Boosting

43 Stacking and Voting

Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

Stacking vs. Voting

- **Voting:**
  - Combine base learners with a fixed rule (majority vote or average).
  - No additional model is trained on top.
- **Stacking:**
  - Learn a meta-model g to combine $\{h_m(x)\}$.
  - Can capture complex relationships between base predictions.
- **Trade-offs:**
  - Stacking is more flexible and often more accurate, but more complex and slower to train.
  - Voting is simpler and easier to interpret.

ltsach@hcmut.edu.vn
Thanh-Sach LE

StackingClassifier in scikit-learn (1/3)

- Setup: Import libraries and create toy dataset.

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

X, y = make_classification(
    n_samples=500, n_features=10,
    n_informative=5, n_redundant=0,
    random_state=0
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

StackingClassifier in scikit-learn (2/3)

- Create base learners (level-0) and meta-learner (level-1).
- Then create `StackingClassifier`.

```
# Base learners (level-0)
clf_lr = LogisticRegression(max_iter=1000, random_state=0)
clf_rf = RandomForestClassifier(n_estimators=100, random_state=0)
clf_svc = SVC(probability=True, random_state=0)

# Meta-learner (level-1)
meta_lr = LogisticRegression(max_iter=1000, random_state=0)

# Create StackingClassifier
from sklearn.ensemble import StackingClassifier

stack_clf = StackingClassifier(
    estimators=[('lr', clf_lr),
                ('rf', clf_rf),
                ('svc', clf_svc)],
    final_estimator=meta_lr,
```

StackingClassifier in scikit-learn (3/3)

- Train models and compare performance.

```
# Fit baseline and stacking models
clf_lr.fit(X_train, y_train)
stack_clf.fit(X_train, y_train)

# Evaluate and compare
print("Logistic Regression acc:",
      accuracy_score(y_test, clf_lr.predict(X_test)))
print("Stacking acc:          ",
      accuracy_score(y_test, stack_clf.predict(X_test)))
```

Expected Output

- Stacking typically achieves higher accuracy than individual base models.
- The meta-learner learns to combine predictions optimally.

Ensemble Methods

Content

Ensemble Methods: Motivation and Overview

Bagging (Bootstrap Aggregating)

Random Forest

Boosting

47 Stacking and Voting

Comparison and Practical Tips

Comparison and Practical Tips

Content

Ensemble Methods: Motivation and Overview

Bagging (Bootstrap Aggregating)

Random Forest

Boosting

Stacking and Voting

48 Comparison and Practical Tips

ltsach@hcmut.edu.vn
Thanh-Sach LE

scikit-learn Ensemble Cheat Sheet

- **Bagging:**
  - Classification: `BaggingClassifier`.
  - Regression: `BaggingRegressor`.
- **Random Forest:**
  - Classification: `RandomForestClassifier`.
  - Regression: `RandomForestRegressor`.
- **Boosting (trees):**
  - AdaBoost: `AdaBoostClassifier`, `AdaBoostRegressor`.
  - Gradient Boosting: `GradientBoostingClassifier`, `GradientBoostingRegressor`.
  - Hist. Grad. Boosting: `HistGradientBoostingClassifier`, `HistGradientBoostingRegressor`.
- **Voting and Stacking:**
  - Voting: `VotingClassifier`, `VotingRegressor`.
  - Stacking: `StackingClassifier`, `StackingRegressor`.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bagging vs Boosting vs Stacking

- **Bagging:**
  - Reduces variance.
  - Works best with high-variance models (e.g., deep trees).
- **Boosting:**
  - Sequentially focuses on hard examples.
  - Can reduce both bias and variance, but may overfit noisy data.
- **Stacking:**
  - Combines heterogeneous models via a meta-learner.
  - More flexible but also more complex to implement and tune.

ltsach@hcmut.edu.vn
Thanh-Sach LE

When to Use Ensemble Methods?

- **Use them when:**
  - Single models underperform on complex datasets.
  - You can afford extra training and prediction time.
- **Be careful about:**
  - Overfitting in boosting with too many iterations or deep trees.
  - Interpretability vs accuracy trade-off.
  - Computational cost on very large datasets.
- **Practical tip:** Random Forest and Gradient Boosting are strong default choices for tabular data.